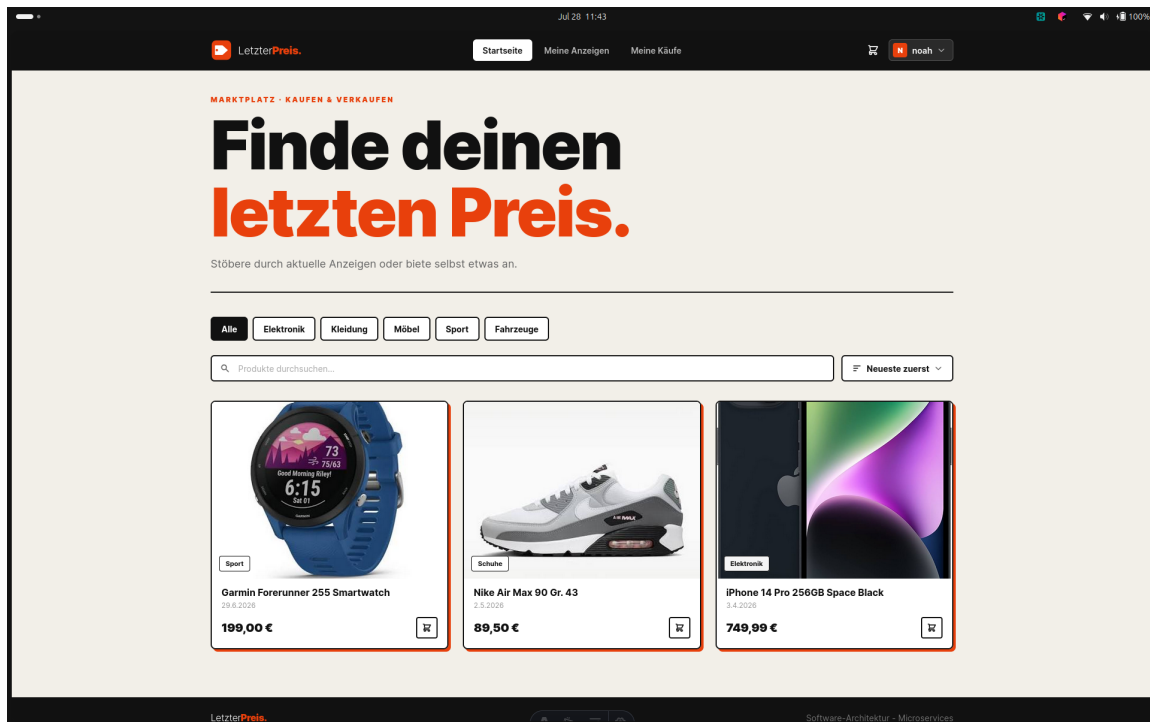




Architekturdokumentation

eBay-Klon als Microservices-Architektur

Gruppe 5 – Software-Architektur

Dominik Duken 21947166

Noah Preyer 35797

Ilker Türk 29963

Juli 2026

Inhaltsverzeichnis

1	Einführung und Ziele	2
1.1	Aufgabenstellung	2
1.2	Kernfunktionen	2
1.3	Qualitätsziele	2
2	Randbedingungen	2
3	Kontextabgrenzung	3
3.1	Fachlicher Kontext	3
3.2	Technischer Kontext	3
3.3	Externe Schnittstellen	3
4	Lösungsstrategie	4
5	Bausteinsicht	4
5.1	Ebene 1: Das Gesamtsystem	4
5.2	Ebene 2 – Order Service (exemplarisch)	6
6	Laufzeitsicht	8
6.1	Registrierung eines Nutzers	8
6.2	Anmeldung und Zugriff auf geschützte Bereiche	8
6.3	Kauf von Artikeln	9
7	Verteilungssicht	11
7.1	Infrastruktur – Ebene 1	11
8	Querschnittliche Konzepte	12
8.1	Sicherheit	12
8.2	Kommunikation	12
8.3	Fehlerbehandlung	12
8.4	Datenhaltung	12
8.5	Serviceübergreifende Konsistenz	12
8.6	Skalierung	13
9	Architekturentscheidungen	13
10	Qualitätsanforderungen	13
11	Risiken und technische Schulden	14
12	Fazit	14
13	Dokumentation der Verwendung von KI-Werkzeugen	15

1 Einführung und Ziele

1.1 Aufgabenstellung

Die Aufgabenstellung bestand darin, einen Architekturstil anhand einer Beispielanwendung umzusetzen. Dabei fiel die Wahl auf die Microservice-Architektur.

Als Beispielanwendung dient eine E-Commerce-Plattform, deren Funktionen sich gut in fachlich getrennte Bereiche aufteilen lassen. Der vollständige Quellcode ist im zugehörigen GitHub-Repository verfügbar:

https://github.com/noah-preyer/SS26_Software_Architektur_Ebay2.git

Die Umsetzung konzentriert sich auf die folgenden Kernfunktionen:

1.2 Kernfunktionen

- Registrierung und Anmeldung mittels JWT
- Erstellen, Bearbeiten, Löschen und Suchen von Produkten
- beispielhafte Kaufabwicklung und E-Mail-Benachrichtigungen

1.3 Qualitätsziele

Qualitätsziel	Beschreibung
Modularität	Die Services sollen fachlich klar voneinander abgegrenzt sein.
Sicherheit	Geschützte Service-Funktionen sollen nur für berechtigte Benutzer zugänglich sein.
Erweiterbarkeit	Neue Services sollen mit möglichst wenigen Änderungen am bestehenden System ergänzt werden können.
Skalierbarkeit	Ein Service soll bei Bedarf unabhängig vom restlichen System durch mehrere Replikate skaliert werden können.
Fehlertoleranz	Der Ausfall eines Services soll andere Bereiche möglichst nicht beeinträchtigen.

Tabelle 1: Qualitätsziele der Architektur

2 Randbedingungen

Der Schwerpunkt des Projekts liegt auf der Umsetzung und Bewertung der Microservice-Architektur. Der Funktionsumfang der Beispielanwendung wurde daher bewusst auf wenige Kernfunktionen begrenzt.

Für die Auswahl der Programmiersprachen, Frameworks und Werkzeuge bestanden keine festen Vorgaben.

3 Kontextabgrenzung

3.1 Fachlicher Kontext

Die E-Commerce-Plattform wird von registrierten und nicht registrierten Benutzern verwendet. Nicht registrierte Benutzer können Produkte suchen und anzeigen sowie ein Benutzerkonto erstellen. Registrierte Benutzer können zusätzlich Produkte einstellen und kaufen.

Nach einem erfolgreichen Kauf wird eine Bestellbestätigung über einen externen E-Mail-Dienst versendet.

3.2 Technischer Kontext

Das System tritt nach außen als eine zusammenhängende Anwendung auf und wird über einen Webbrowser genutzt. Das Frontend kommuniziert über HTTP/REST mit dem API Gateway, das den zentralen Einstiegspunkt bildet.

Die interne Kommunikation zwischen den Services erfolgt über REST und MQTT. Die externen Schnittstellen sind in Tabelle 2 dargestellt.

3.3 Externe Schnittstellen

Gegenstelle	Richtung	Inhalt und Protokoll
Webbrowser	Bidirektional	Bedienung der Plattform über HTTP/REST.
SMTP-Server	Ausgehend	Versand von Bestellbestätigungen über SMTP mit STARTTLS auf Port 587.

Tabelle 2: Externe Schnittstellen

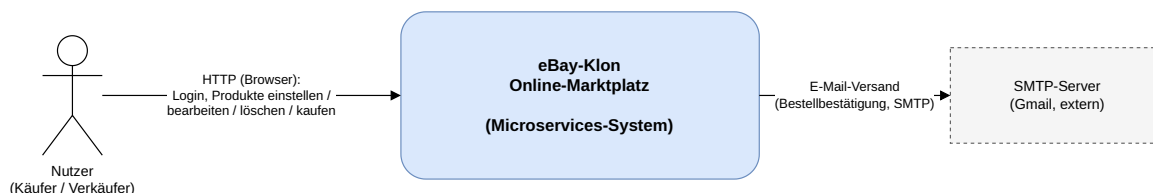


Abbildung 1: Kontextdiagramm der E-Commerce-Plattform

4 Lösungsstrategie

Zur Umsetzung der Microservice-Architektur wurden folgende zentrale Architekturmaßnahmen gewählt:

Architekturmaßnahme	Umsetzung
Systemaufteilung	Aufteilung in fachlich getrennte Microservices.
Betrieb	Containerisierung der Services mit Docker.
Zugriff	Ein zentrales API Gateway bündelt die externen Zugriffe und validiert die JWTs.
Kommunikation	REST für synchrone Serviceaufrufe und MQTT für asynchrone Bestellereignisse.
Datenkonsistenz	Kompensierende Rückabwicklung statt serviceübergreifender Datenbanktransaktionen.
Technologien	Einsatz von Spring Boot, Flask, Astro und SolidJS.
Skalierung	Zustandslose Services können unabhängig repliziert werden. Der Product Service läuft exemplarisch mit drei Replikaten hinter Nginx.

Tabelle 3: Zentrale Architekturmaßnahmen

5 Bausteinsicht

5.1 Ebene 1: Das Gesamtsystem

Das System besteht aus sieben eigenständig ausgelieferten Bausteinen sowie einem Nachrichtenbroker und einem Loadbalancer. Die Aufteilung orientiert sich an den fachlichen Verantwortungsbereichen des Systems. Abbildung 2 zeigt den Aufbau des Systems.

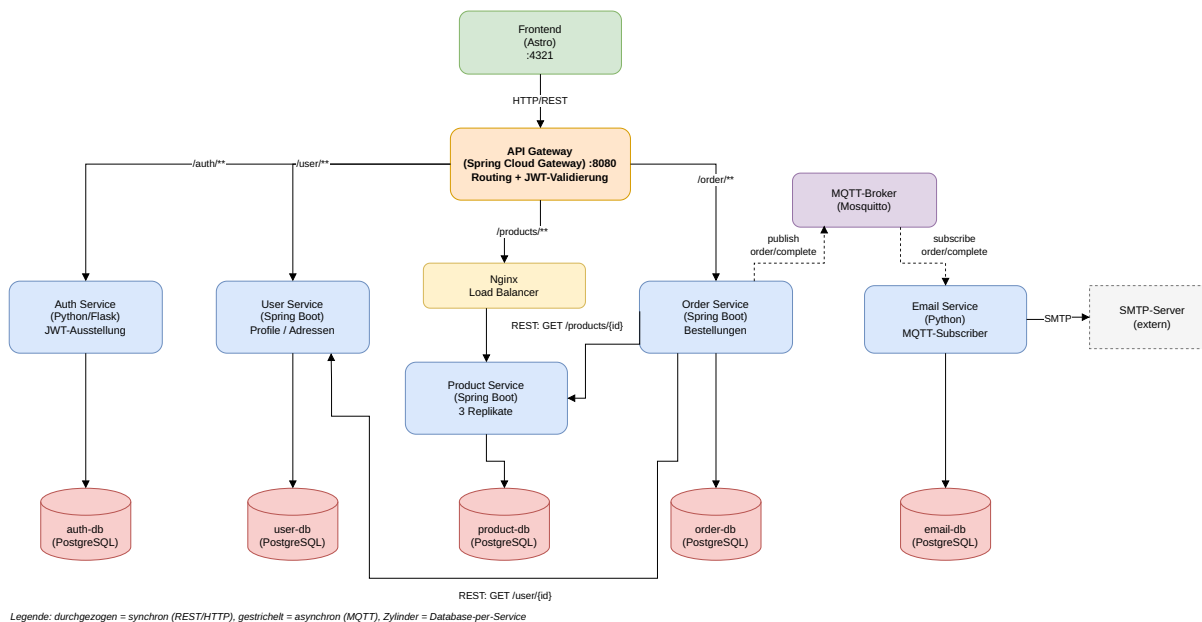


Abbildung 2: Bausteinsicht

Die Verantwortlichkeiten der einzelnen Bausteine sind in Tabelle 4 zusammengefasst.

Tabelle 4: Verantwortlichkeiten der Bausteine

Baustein	Verantwortung
Frontend	Mit Astro und SolidJS umgesetzt. Stellt Komponenten und Seiten für Produkte, Warenkorb und Benutzerkonto bereit.
API Gateway	Als Spring-Cloud-Gateway-Anwendung umgesetzt. Enthält die Routen zu den Services sowie einen Filter zur JWT-Validierung.
Auth Service	Als Flask-Anwendung umgesetzt. Enthält REST-Endpunkte für Registrierung und Anmeldung sowie die Erzeugung von JWTs.
User Service	Als Spring-Boot-Anwendung mit Controller-, Service- und Repository-Schicht umgesetzt. Verwaltet Benutzerprofile und Lieferadressen.
Product Service	Als Spring-Boot-Anwendung mit Controller-, Service- und Repository-Schicht umgesetzt. Verwaltet Produkte und deren Verkaufstatus.
Order Service	Als Spring-Boot-Anwendung umgesetzt. Koordiniert den Kaufvorgang über REST-Clients, speichert Bestellungen und veröffentlicht Bestellereignisse über MQTT.
Email Service	Als Python-Anwendung mit <code>paho-mqtt</code> umgesetzt. Abonniert Bestellereignisse, rendert E-Mail-Vorlagen und versendet Bestellbestätigungen.
MQTT-Broker	Vermittelt die asynchronen Bestellereignisse zwischen Order Service und Email Service.

Fortsetzung auf der nächsten Seite

Tabelle 4 – Fortsetzung

Baustein	Verantwortung
Nginx	Verteilt Produktanfragen im Round-Robin-Verfahren auf die drei Instanzen des Product Service.

5.2 Ebene 2 – Order Service (exemplarisch)

Der Order Service ist intern in Controller, Geschäftslogik, Datenzugriff und ausgehende Clients gegliedert. Die ausgehenden REST- und MQTT-Aufrufe sind in eigenen Client-Klassen gekapselt. Dadurch bleibt die eigentliche Kaufabwicklung im **OrderService** weitgehend unabhängig von den verwendeten Kommunikationsprotokollen.

Tabelle 5 beschreibt die wichtigsten Codebausteine und Abbildung 3 zeigt deren Abhängigkeiten.

Tabelle 5: Codebausteine des Order Service

Codebaustein	Aufgabe
OrderController	Stellt die REST-Endpunkte des Order Service bereit, nimmt Anfragen entgegen und delegiert die Verarbeitung an den OrderService .
OrderService	Enthält die Geschäftslogik des Kaufvorgangs. Prüft Produkte, reserviert Artikel, legt Bestellungen an und löst die Bestellbestätigung aus.
OrderRepository	Kapselt mit Spring Data JPA den Zugriff auf die order-service-db .
ProductServiceClient	Kapselt die REST-Aufrufe zum Product Service, beispielsweise zum Abrufen, Reservieren und Freigeben eines Produkts.
UserServiceClient	Ruft die Benutzerdaten und die E-Mail-Adresse des Käufers beim User Service ab.
EmailServiceClient	Veröffentlicht nach einem erfolgreichen Kauf ein MQTT-Ereignis auf dem Topic order/complete .
OrderStatusScheduler	Sucht regelmäßig nach fälligen Bestellungen und führt die Statusübergänge CREATED , PAID , SHIPPED und DELIVERED aus.
GlobalExceptionHandler	Wandelt Exceptions in einheitliche HTTP-Fehlerantworten um.
DTOs und Domänenklassen	Enthalten die internen Bestellobjekte sowie die Datenmodelle für die Kommunikation mit Product Service und User Service.

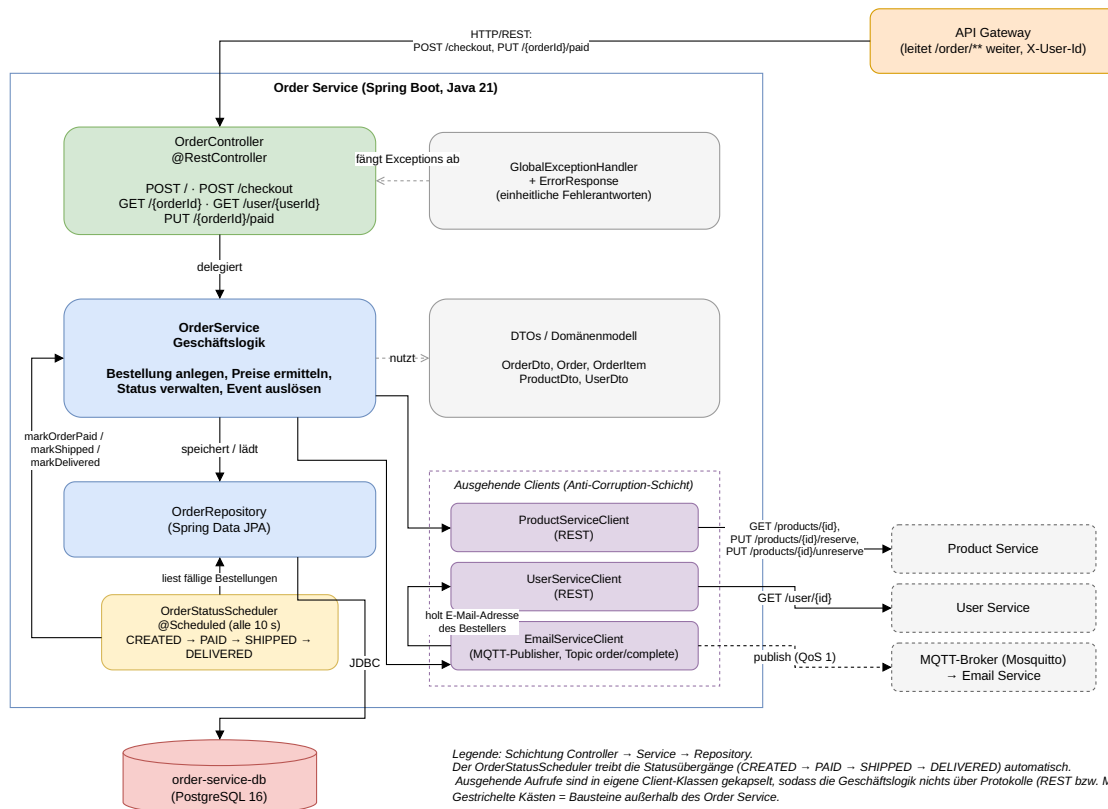


Abbildung 3: Order Service

6 Laufzeitsicht

6.1 Registrierung eines Nutzers

Das API Gateway leitet die Registrierungsdaten an den User Service weiter. Dieser prüft, ob Benutzername und E-Mail-Adresse bereits vergeben sind, und legt anschließend die Zugangsdaten über den Auth Service an. Danach wird das Benutzerprofil gespeichert.

Schlägt das Speichern des Profils fehl, wird der zuvor angelegte Auth-Benutzer durch einen kompensierenden Aufruf wieder gelöscht. Der Ablauf ist in Abbildung 4 dargestellt.

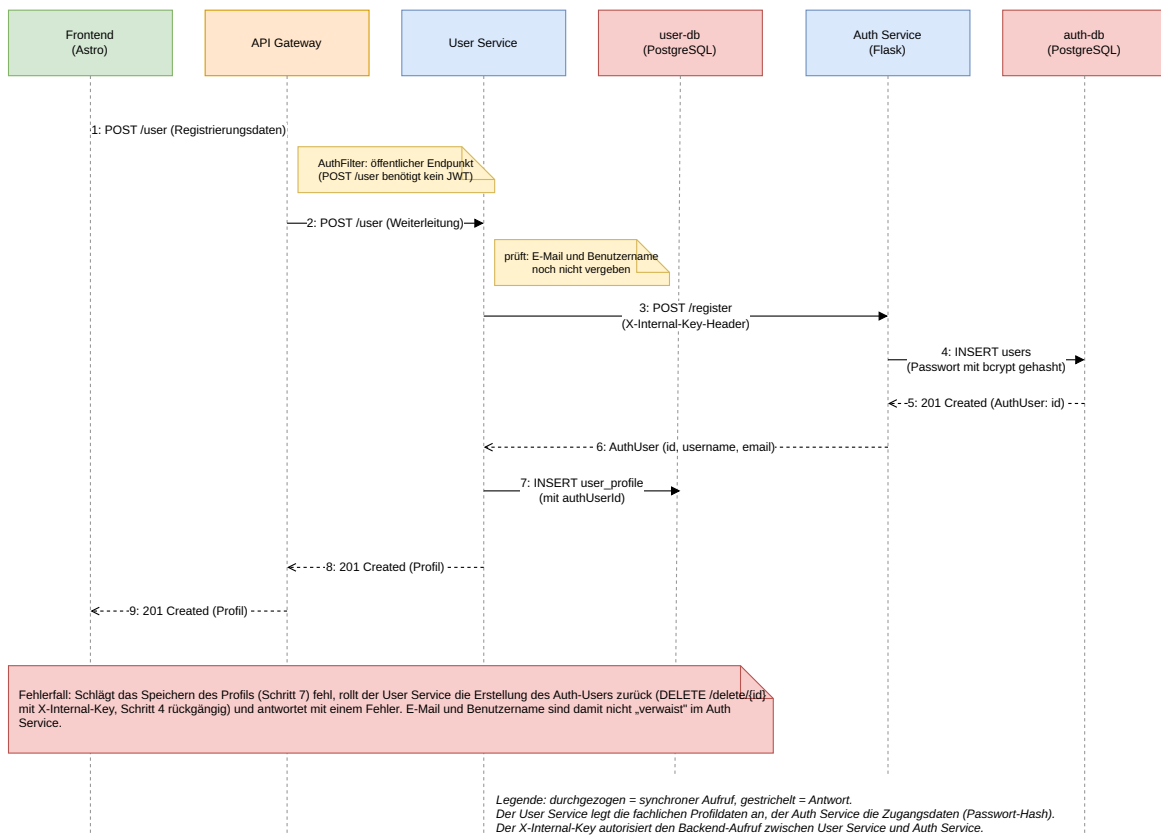


Abbildung 4: Registrierung eines Nutzers

6.2 Anmeldung und Zugriff auf geschützte Bereiche

Bei der Anmeldung leitet das API Gateway die Zugangsdaten an den Auth Service weiter. Nach erfolgreicher Prüfung stellt dieser ein 60 Minuten gültiges JWT aus.

Bei geschützten Aufrufen validiert das API Gateway das Token. Ungültige oder abgelaufene Tokens werden mit 401 **Unauthorized** abgewiesen. Andernfalls wird die Benutzer-ID aus dem JWT-Claim ausgelesen und als **X-User-Id-Header** an den zuständigen Service weitergegeben. Der vollständige Ablauf ist in Abbildung 5 dargestellt.

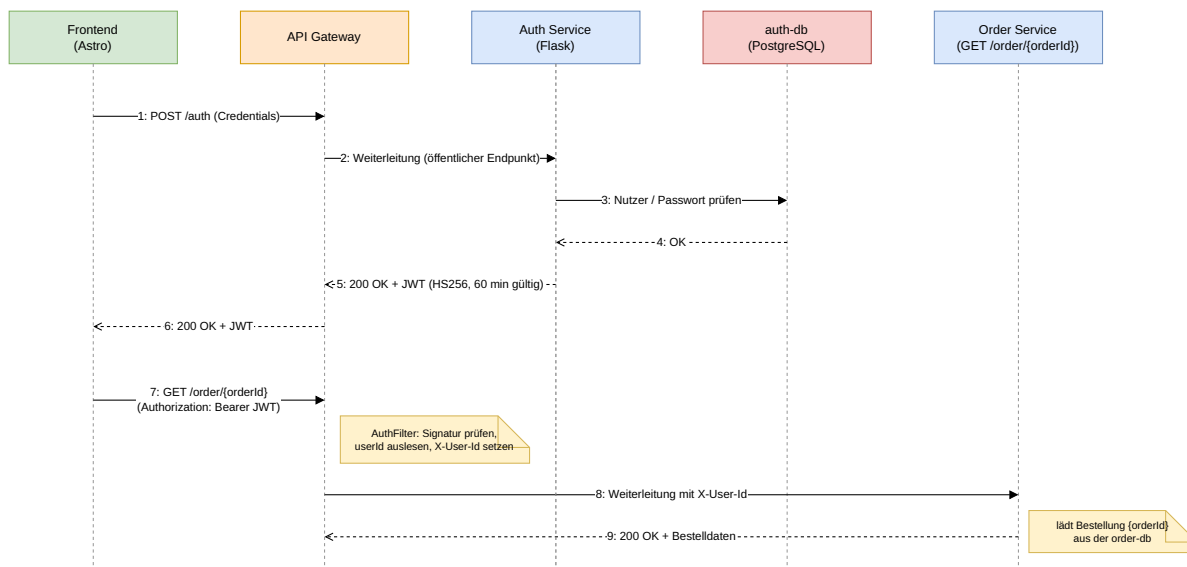


Abbildung 5: Anmeldung und anschließender Zugriff mit Token

6.3 Kauf von Artikeln

Beim Kauf übermittelt das Frontend alle Artikel des Warenkorbs in einer Anfrage. Der Order Service prüft beim Product Service, ob die Artikel noch verfügbar sind und nicht dem Käufer selbst gehören. Schlägt eine Prüfung fehl, wird der gesamte Kauf abgebrochen.

Anschließend markiert der Order Service die Artikel als verkauft. Tritt dabei ein Fehler auf, werden bereits vorgenommene Änderungen zurückgesetzt. Danach wird die Bestellung im Status **CREATED** gespeichert. Die Benutzer-ID bezieht der Order Service aus dem **X-User-Id**-Header, den das API Gateway aus dem JWT gesetzt hat; mit ihr wird die E-Mail-Adresse beim User Service abgerufen (**GET /user/by-auth/{authUserId}**), um die Bestellbestätigung zusammenzustellen.

Der Order Service veröffentlicht über MQTT ein Ereignis, das vom Email Service unabhängig verarbeitet wird. Dadurch kann die Bestellbestätigung versendet werden, ohne die Antwort an den Benutzer zu verzögern.

Die weiteren Statusübergänge von **CREATED** über **PAID** und **SHIPPED** bis **DELIVERED** werden automatisch vom Order Status Scheduler durchgeführt. Der vollständige Ablauf ist in Abbildung 6 dargestellt.

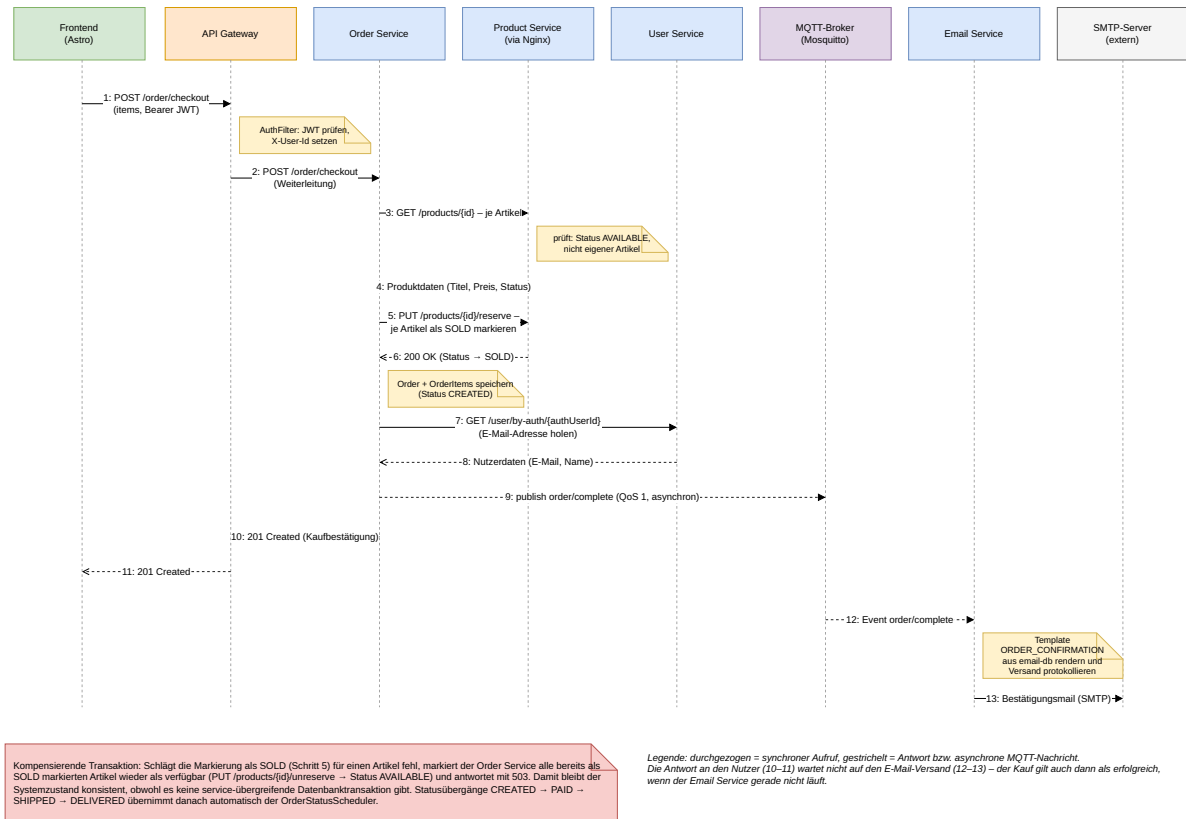


Abbildung 6: Kauf von Artikeln und Versand der Bestellbestätigung

7 Verteilungssicht

7.1 Infrastruktur – Ebene 1

Das Gesamtsystem wird mit Docker Compose betrieben. Alle Container befinden sich im gemeinsamen Docker-Netzwerk `app-net`.

Nur das Frontend veröffentlicht mit Port 4321 einen Port auf dem Docker-Host. Es leitet Backend-Anfragen intern über `http://api-gateway:8080` an das API Gateway weiter.

Die Services, Datenbanken und der MQTT-Broker sind ausschließlich innerhalb des Docker-Netzwerks erreichbar. Der Product Service wird mit drei Replikaten ausgeführt, auf die ein interner Nginx-Loadbalancer die Anfragen verteilt.

Container	Host-Port	Interner Port	Anmerkung
Frontend	4321	4321	Einzigster von außen erreichbarer Container.
API Gateway	–	8080	Zentraler Backend-Einstieg, nur intern erreichbar.
Auth, User und Order Service	–	8080	Nur innerhalb von <code>app-net</code> erreichbar.
Product Service	–	8082	Drei Replikate, nur intern erreichbar.
Product-Service-Proxy	–	80	Interner Nginx-Loadbalancer.
Email Service	–	–	Kommunikation ausschließlich über MQTT.
MQTT-Broker	–	1883, 9001	Nur innerhalb von <code>app-net</code> erreichbar.
PostgreSQL-Datenbanken	–	5432	Je zustandsbehaftetem Service eine Instanz.

Tabelle 6: Container- und Portübersicht

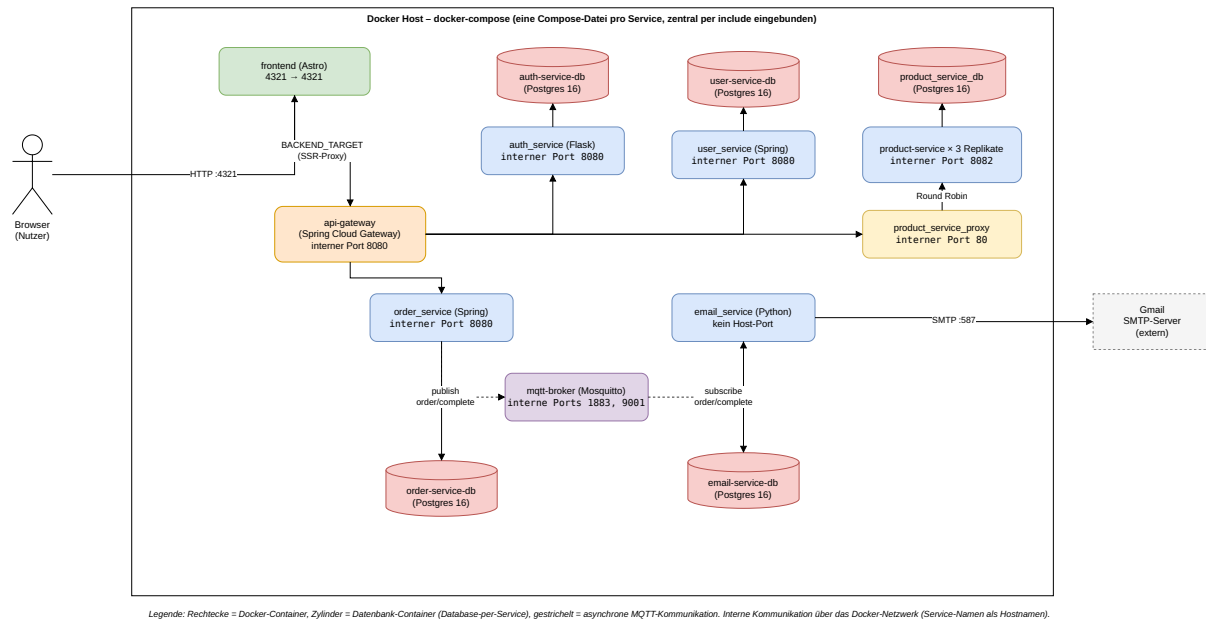


Abbildung 7: Deployment-Diagramm

8 Querschnittliche Konzepte

Die folgenden technischen Regeln gelten serviceübergreifend.

8.1 Sicherheit

Der Auth Service stellt JWTs mit einer Gültigkeit von 60 Minuten aus. Das API Gateway validiert die Tokens und übergibt die Benutzer-ID über den Header `X-User-Id`. Der interne Aufruf des User Service an den Auth Service wird zusätzlich durch den Header `X-Internal-Key` autorisiert.

8.2 Kommunikation

Synchrone Serviceaufrufe erfolgen über REST. Bestellereignisse werden über MQTT mit Zustellbestätigung übertragen.

8.3 Fehlerbehandlung

Die Spring-basierten Services verwenden ein einheitliches Fehlerformat. Validierungsfehler liefern 400, fehlende Ressourcen 404, Konflikte 409 und unerwartete Fehler 500.

8.4 Datenhaltung

Jeder zustandsbehaftete Service besitzt eine eigene PostgreSQL-Datenbank. Nur der jeweilige Service darf direkt auf seine Datenbank zugreifen. Serviceübergreifende Beziehungen werden über IDs statt über datenbankübergreifende Fremdschlüssel abgebildet.

8.5 Serviceübergreifende Konsistenz

Serviceübergreifende Datenbanktransaktionen werden nicht verwendet. Schlägt die Änderung eines Produkts während des Kaufvorgangs fehl, setzt der Order Service bereits vorgenommene Änderungen

durch kompensierende Aufrufe zurück.

8.6 Skalierung

Die Instanzen des Product Service speichern keine Sitzungsdaten oder lokalen Dateien. Dadurch können Anfragen über Nginx im Round-Robin-Verfahren auf drei Replikate verteilt werden.

Alle Replikate greifen auf dieselbe `product-db` zu. Die Datenbank kann daher bei hoher Last zum Engpass werden. Eine automatische, lastabhängige Skalierung ist nicht umgesetzt; die Anzahl der Replikate wird beim Start festgelegt.

9 Architekturentscheidungen

Die folgenden Entscheidungen prägen den Aufbau und die Implementierung des Systems. Neben der Begründung werden auch die daraus entstehenden Nachteile und zusätzlichen Aufwände betrachtet.

Nr.	Entscheidung	Begründung	Konsequenz
AE 1	Fachliche Aufteilung in eigenständige Services	Authentifizierung, Benutzerverwaltung, Produkte, Bestellungen und Benachrichtigungen besitzen getrennte Verantwortlichkeiten und sollen unabhängig entwickelt werden können.	Die Services können getrennt entwickelt und betrieben werden. Dafür entstehen zusätzlicher Deployment-Aufwand, Netzwerkkommunikation und verteilte Fehlerfälle.
AE 2	Zentrales API Gateway	Routing und JWT-Prüfung sollen nicht in jedem Service einzeln implementiert werden.	Das Gateway bildet einen einheitlichen Zugang, verursacht aber einen zusätzlichen Networkhop und ist ein zentraler Ausfallpunkt.
AE 3	Eigene Datenbank je Service	Jeder Service soll alleiniger Eigentümer seiner Daten und seines Datenmodells sein.	Die Services bleiben hinsichtlich ihrer Datenhaltung unabhängig. Serviceübergreifende Fremdschlüssel und gemeinsame Transaktionen sind jedoch nicht möglich.
AE 4	Kompensation statt verteilter Transaktion	Product Service und Order Service besitzen getrennte Datenbanken, sodass keine gemeinsame Datenbanktransaktion verwendet werden kann.	Bereits geänderte Produkte werden bei einem Fehler durch Gegenaufrufe wieder freigegeben. Diese Rückabwicklung erfordert zusätzliche Programmlogik.
AE 5	Zustandslose Product-Service-Replikate	Produktanfragen sollen exemplarisch auf mehrere Instanzen verteilt werden können.	Drei Product-Service-Instanzen können über Nginx angesprochen werden. Die gemeinsam verwendete Produktdatenbank bleibt jedoch ein möglicher Engpass.

10 Qualitätsanforderungen

Die in Kapitel 1 definierten Qualitätsziele werden anhand konkreter Szenarien bewertet.

Qualitätsziel	Szenario	Bewertung
Modularität	Produktverwaltung, Benutzerverwaltung und Bestellabwicklung sind in eigenständigen Services mit klar abgegrenzten Verantwortlichkeiten gekapselt.	Erfüllt
Sicherheit	Aufrufe geschützter Endpunkte ohne gültiges JWT werden am API Gateway abgewiesen.	Erfüllt
Erweiterbarkeit	Ein neuer Service kann über eine eigene Compose-Konfiguration und eine neue Gateway-Route ergänzt werden.	Weitgehend erfüllt
Skalierbarkeit	Produktanfragen werden auf drei Instanzen des Product Service verteilt. Die Anzahl der Instanzen ist jedoch statisch.	Teilweise erfüllt
Fehlertoleranz	Der Ausfall des Email Service soll den erfolgreichen Abschluss eines Kaufvorgangs nicht verhindern.	Erfüllt

Tabelle 7: Bewertung der Qualitätsziele

11 Risiken und technische Schulden

API Gateway als Single Point of Failure Bei einem Ausfall des API Gateways ist das gesamte System nicht mehr erreichbar. Ein Failover-Mechanismus ist nicht vorhanden.

Synchrone Serviceabhängigkeiten Der Order Service ruft den Product Service und den User Service synchron über REST auf. Sind diese nicht erreichbar, kann der Kaufvorgang nicht abgeschlossen werden.

Keine automatische Skalierung Die Anzahl der Product-Service-Replikate ist fest konfiguriert und wird nicht automatisch an die aktuelle Last angepasst.

12 Fazit

Die Umsetzung verdeutlicht sowohl die Vorteile als auch die Nachteile einer Microservice-Architektur. Die fachliche Aufteilung, klar definierte Schnittstellen und getrennte Datenbanken ermöglichen eine weitgehend unabhängige Entwicklung der einzelnen Services. Durch die Kombination aus synchroner und asynchroner Kommunikation konnten unterschiedliche Formen der Servicekopplung praktisch umgesetzt und erprobt werden.

Gleichzeitig erhöht die Verteilung der Anwendung den technischen Aufwand. Betrieb und Fehlersuche erstrecken sich über mehrere Services, Container und Kommunikationswege. Serviceübergreifende Vorgänge können nicht durch eine gemeinsame Datenbanktransaktion abgesichert werden und benötigen deshalb zusätzliche Kompensationslogik. Zudem bleibt der Kaufvorgang von synchron aufgerufenen Services wie dem Product Service und dem User Service abhängig.

Für größere Systeme mit mehreren Entwicklungsteams, unterschiedlich belasteten Teilbereichen oder hohen Anforderungen an die Fehlerisolation kann eine Microservice-Architektur sinnvoll sein. Für den begrenzten Umfang der umgesetzten Beispielanwendung wäre ein modularer Monolith einfacher zu entwickeln und zu betreiben gewesen. Als Lernprojekt war die gewählte Architektur dennoch geeignet, da ihre Auswirkungen auf Implementierung, Kommunikation, Datenhaltung und Fehlerbehandlung praktisch untersucht werden konnten.

13 Dokumentation der Verwendung von KI-Werkzeugen

Zweck der Verwendung	Eingesetztes KI-Werkzeug	Verwendungsweise bzw. -ort
Überarbeitung von Formulierungen und Schreibstil	ChatGPT GPT-5.6 SOL	Punktuelle Vorschläge für alternative Formulierungen sowie sprachliche und stilistische Überarbeitungen einzelner Textabschnitte
Unterstützung bei der Entwicklung mit Java und Spring Boot	Claude Code, ChatGPT GPT-5.6 SOL, DeepSeek V4 Flash Free	Punktuelle Unterstützung bei der Fehlersuche, der Überprüfung bestehender Implementierungen und der Ausarbeitung einzelner Codevorschläge für die Java-Spring-Boot-Services
Unterstützung bei der Entwicklung mit Python	ChatGPT GPT-5.6 SOL, DeepSeek V4 Flash Free	Punktuelle Unterstützung bei der Fehlersuche, der Anpassung bestehender Implementierungen und der Ausarbeitung einzelner Codevorschläge für die Python-Komponenten
Unterstützung bei der Frontend-Entwicklung mit Astro und SolidJS	Claude Code	Punktuelle Unterstützung bei der Fehlersuche, der Überprüfung bestehender Komponenten und der Ausarbeitung einzelner Lösungsvorschläge für Astro, SolidJS, JavaScript, HTML und CSS

Tabelle 8: KI-Werkzeugen